Software Testing and Quality Assurance

System Testing on Jira-Lite API

Submitted By

Group USLS (A9)

011221444 Atique Bin Mahmud

011221520 Ishrak Ahmed

011221521 Ilham Mahfuz Riyadh

011223697 Tony Roy

Submitted To

Mobaswirul Islam

Lecturer

Dept. of CSE

Date of Submission: 12th April, 2026

1. Introduction

This document provides a complete test report for the API system that replicates the functionalities of Jira. During testing, a well-articulated approach was followed to verify the validity of all API system functionalities, including user authentication, team management, project creation, issue lifecycle management, and commenting.

The system includes functionalities of a full-blown issue tracking system such as registering users, creating teams, creation of projects (both limited and open), managing issues in their entire lifecycles (from being opened to being closed), commenting on issues, role-based permissions, and maintaining audit trail through activity logs and resolve history.

The test was done through Postman using a character-based approach by defining four characters (Alice, Bob, Charlie, and David). **122 test cases** were carried out during the process that included both positive and negative cases, as well as edge cases. Through a systematic process of testing, **14 defects** were discovered, and details have been provided within this report.

2. Test Approach & Methodology

2.1 Character-Based Testing Strategy

To ensure comprehensive coverage of multi-user scenarios and permission validations, the following character personas were established:

Character	Role	Purpose
Alice	Team Alpha Leader, Project Creator	Tests team leadership privileges, project ownership, and issue creator permissions
Bob	Team Alpha Member	Validates team member capabilities, issue resolution workflows
Charlie	Team Beta Leader (Different Team)	Tests cross-team restrictions and independent team operations
David	Independent User (No Team)	Validates open project access and non-team-member restrictions

2.2 Testing Workflow

The testing followed a sequential dependency chain:

Step 1: User Registration & Authentication (Alice, Bob, Charlie, David)

Step 2: Team Formation (Team Alpha by Alice, Team Beta by Charlie)

Step 3: Team Member Management (Adding/Removing members)

Step 4: Project Creation (Team-restricted & Open projects)

Step 5: Issue Creation & Lifecycle Testing

Step 6: Comment System Validation

This approach ensured that prerequisite conditions were properly established before executing dependent test cases, maintaining data integrity throughout the testing process.

3. Test Plan

3.1 Authentication Endpoints

Endpoint 01: POST /auth/register

Aspect	Details
Input / Parameter Choices	username (string), email (string), password (string, min 6 chars)
Expected Output	On valid input: User object with id, username, email, createdAt. On invalid input: Error message with specific failure reason. Password hash never returned.
Expected HTTP Status	201 Created-User successfully registered 400 Bad Request - Missing fields or password too short 409 Conflict- Email or username already exists

Endpoint 02: POST /auth/login

Aspect	Details
Input / Parameter Choices	email (string), password (string)
Expected Output	On valid credentials: JWT token. On invalid: Error message.
Expected HTTP Status	200 OK-Login successful, token returned 400 Bad Request-Missing email or password 401 Unauthorized -Invalid credentials

3.2 User Management Endpoints

Endpoint 03: GET /users/:id

Aspect	Details
Input / Parameter Choices	id (UUID in URL path), Authorization header with Bearer token
Expected Output	User object containing id, username, email, createdAt. Password/hash fields excluded.
Expected HTTP Status	200 OK- User found and returned, 401 Unauthorized - Missing or invalid token 404 Not Found - User ID does not exist

Endpoint 04: PATCH /users/profile

Aspect	Details
Input / Parameter Choices	Optional fields: username, email, password. Must be authenticated user's own profile.
Expected Output	Updated user object with modified fields.
Expected HTTP Status	200 OK - Profile updated successfully 401 Unauthorized - Missing token 409 Conflict - Username or email already taken by another user

3.3 Team Management Endpoints

Endpoint 05: POST /teams

Aspect	Details
Input / Parameter Choices	name (string, required). Authenticated user becomes leader and first member.
Expected Output	Team object with id, name, leaderId, memberIds (array), createdAt.
Expected HTTP Status	201 Created - Team successfully created 400 Bad Request - Missing name 401 Unauthorized - Missing token

Endpoint 06: GET /teams/:id

Aspect	Details
Input / Parameter Choices	id (UUID in URL path), Authorization required
Expected Output	Complete team object with all members.
Expected HTTP Status	200 OK - Team found 401 Unauthorized - Missing token 404 Not Found - Team does not exist

Endpoint 07: POST /teams/:id/members

Aspect	Details
Input / Parameter Choices	userId (UUID, required). Only team leader can add members.
Expected Output	Updated team object with new member in memberIds array.
Expected HTTP Status	200 OK- Member added successfully 400 Bad Request - Missing userId 401 Unauthorized - Missing token 403 Forbidden - Requester is not team leader 404 Not Found - Team or user not found 409 Conflict - User already a member

Endpoint 08: DELETE /teams/:id/members/:userId

Aspect	Details
Input / Parameter Choices	id (team UUID), userId (member UUID). Only leader can remove. Cannot remove leader.
Expected Output	Updated team object with member removed from memberIds.
Expected HTTP Status	200 OK - Member removed successfully 400 Bad Request - Attempting to remove leader 401 Unauthorized - Missing token 403 Forbidden - Not team leader 404 Not Found - Team not found or user not a member

3.4 Project Management Endpoints

Endpoint 09: POST /projects

Aspect	Details
Input / Parameter Choices	name (string, required), teamId (UUID, optional). If teamId provided, user must be team member.
Expected Output	Project object with id, name, creatorId, teamId (null for open projects), createdAt.
Expected HTTP Status	201 Created - Project created 400 Bad Request - Missing name 401 Unauthorized - Missing token 403 Forbidden - Not a member of specified team 404 Not Found - Team not found

Endpoint 10: GET /projects

Aspect	Details
Input / Parameter Choices	Optional query parameter: teamId (UUID) for filtering. Authorization required.
Expected Output	Array of project objects. Empty array if no matches.
Expected HTTP Status	200 OK - Projects returned (may be empty array) 401 Unauthorized - Missing token

Endpoint 11: GET /projects/:id

Aspect	Details
Input / Parameter Choices	id (UUID in path). Authorization required.

Expected Output	Single project object with all details.
Expected HTTP Status	200 OK - Project found 401 Unauthorized - Missing token 404 Not Found - Project does not exist

Endpoint 12: POST /projects/:projectId/issues

Aspect	Details
Input / Parameter Choices	title (string, required), description (string, required). Any authenticated user can create issues.
Expected Output	Issue object with id, projectId, creatorId, title, description, status (open), resolverId (null), empty resolveHistory, activityLog with "opened" entry, reopenCount (0).
Expected HTTP Status	201 Created - Issue created 400 Bad Request - Missing title or description 401 Unauthorized - Missing token 404 Not Found - Project not found

Endpoint 13: GET /projects/:projectId/issues

Aspect	Details
Input / Parameter Choices	projectId (UUID in path). Authorization required.
Expected Output	Array of all issues belonging to the project.
Expected HTTP Status	200 OK - Issues returned 401 Unauthorized - Missing token 404 Not Found - Project not found

3.5 Issue Management Endpoints

Endpoint 14: GET /issues

Aspect	Details
Input / Parameter Choices	Optional filters: status, creatorId, creatorName, resolverId, resolverName, teamId, teamName. All case-insensitive for name filters.
Expected Output	Array of filtered issues. Empty array if no matches or name-based filter not found.
Expected HTTP Status	200 OK - Filtered results returned 401 Unauthorized - Missing token

Endpoint 15: GET /issues/:id

Aspect	Details
Input / Parameter Choices	id (UUID in path). Authorization required.
Expected Output	Complete issue object including resolveHistory and activityLog arrays.
Expected HTTP Status	200 OK - Issue found 401 Unauthorized - Missing token 404 Not Found - Issue does not exist

Endpoint 16: PATCH /issues/:id/start

Aspect	Details
Input / Parameter Choices	id (UUID). For team projects: must be team member. For open projects: any authenticated user. Issue must be in "open" status.
Expected Output	Issue with status changed to "in_progress", resolverId set to current user, new entry in resolveHistory with startedAt timestamp.
Expected HTTP Status	200 OK - Issue started 400 Bad Request - Issue not in open status 401 Unauthorized - Missing token 403 Forbidden - Not a team member (for team projects) 404 Not Found - Issue not found

Endpoint 17: PATCH /issues/:id/abandon

Aspect	Details
Input / Parameter Choices	id (UUID). Only current resolver can abandon. Issue must be "in_progress".
Expected Output	Issue with status changed to "open", resolverId set to null, resolveHistory updated with outcome: "abandoned" and endedAt timestamp.
Expected HTTP Status	200 OK - Issue abandoned 400 Bad Request - Issue not in_progress 401 Unauthorized - Missing token 403 Forbidden - Not current resolver 404 Not Found - Issue not found

Endpoint 18: PATCH /issues/:id/submit

Aspect	Details
Input / Parameter Choices	id (UUID). Only current resolver can submit. Issue must be "in_progress".
Expected Output	Issue with status changed to "resolved", resolveHistory updated with outcome: "submitted", resolvedBy set to resolver, endedAt timestamp.
Expected HTTP Status	200 OK - Solution submitted 400 Bad Request Issue not in_progress 401 Unauthorized Missing token 403 Forbidden - Not current resolver 404 Not Found - Issue not found

Endpoint 19: PATCH /issues/:id/accept

Aspect	Details
Input / Parameter Choices	id (UUID). Only issue creator can accept. Issue must be "resolved".
Expected Output	Issue with status changed to "closed", resolveHistory updated with outcome: "accepted".
Expected HTTP Status	200 OK - Solution accepted 400 Bad Request - Issue not resolved 401 Unauthorized - Missing token 403 Forbidden - Not issue creator 404 Not Found - Issue not found

Endpoint 20: PATCH /issues/:id/reopen

Aspect	Details
Input / Parameter Choices	id (UUID). Only issue creator can reopen. Issue must be "resolved" or "closed".
Expected Output	Issue with status changed to "open", resolverId set to null, reopenCount incremented, resolveHistory updated with outcome: "reopened".
Expected HTTP Status	200 OK - Issue reopened 400 Bad Request - Issue not resolved/closed 401 Unauthorized - Missing token403 Forbidden - Not issue creator 404 Not Found – Issue not found

3.6 Comment Endpoints

Endpoint 21: POST /issues/:issueId/comments

Aspect	Details
Input / Parameter Choices	body (string, required), parentCommentId (UUID, optional). Cannot comment on closed issues. Parent must belong to same issue.
Expected Output	Comment object with id, issueId, authorId, body, parentCommentId, createdAt.
Expected HTTP Status	201 Created - Comment posted 400 Bad Request - Missing body, issue closed, or parent from different issue 401 Unauthorized - Missing token 404 Not Found - Issue or parent comment not found

Endpoint 22: GET /issues/:issueId/comments

Aspect	Details
Input / Parameter Choices	issueId (UUID in path). Authorization required.
Expected Output	Nested tree structure with top-level comments and recursive replies arrays.
Expected HTTP Status	200 OK - Comments returned as tree 401 Unauthorized - Missing token 404 Not Found - Issue not found

4. Test Cases

Test Case Summary

Category	Total Tests	Passed	Failed
Authentication	9	8	1
User Management	7	6	1
Team Management	20	17	3
Project Management	18	18	0
Issue Management	43	35	8
Comments	17	16	1
Setup Tests	8	8	0
TOTAL	122	108	14

4.1 Detailed Test Cases

Test ID	Title	Pre-conditions	Test Steps	Expected Result	Actual Result	Status	Evidence	Pass/Fail
TC-001	Register User - Success (Alice)	Server running, no existing user with email/username	1. POST /auth/register 2. Body: {"username":"alice","email":"alice@example.com","password":"pass123"}	201 Created, user object with id, username, email returned. No password field.	As expected	User created successfully	TC-1.png	PASS
TC-002	Register - Missing Field	Server running	1. POST /auth/register 2. Body: {"username":"testuser","email":"test@example.com"} (password missing)	400 Bad Request, error: "username, email and password are required"	As expected	Validation working	TC-2.png	PASS
TC-003	Register - Email Already Used	Alice registered	1. POST /auth/register 2. Body: {"username":"alice2","email":"alice@example.com","password":"pass123"}	409 Conflict, error: "Email already in use"	As expected	Duplicate email prevented	TC-3.png	PASS
TC-004	Register - Username Taken	Alice registered	1. POST /auth/register 2. Body: {"username":"alice","email":"alice2@example.com","password":"pass123"}	409 Conflict, error: "Username already taken"	As expected	Duplicate username prevented	TC-4.png	PASS
TC-005	Register - Short Password	Server running	1. POST /auth/register 2. Body: {"username":"newuser","email":"new@example.com","password":"123"}	400 Bad Request, error: "Password must be at least 6 characters long"	As expected	Password validation working	TC-5.png	PASS

TC-006	Login - Success (Alice)	Alice registered	1. POST /auth/login 2. Body: {"email":"alice@example.com","password":"pass123"}	200 OK, token returned	As expected	Login successful, token saved	TC-6.png	PASS
TC-007	Login - Missing Field	Server running	1. POST /auth/login 2. Body: {"email":"alice@example.com"} (password missing)	400 Bad Request, error: "email and password are required"	As expected	Validation working	TC-7.png	PASS
TC-008	Login - Email Not Found	Server running	1. POST /auth/login 2. Body: {"email":"nonexistent@example.com","password":"pass123"}	401 Unauthorized, error: "Invalid credentials"	As expected	Non-existent email handled	TC-8.png	PASS
TC-009	Login - Wrong Password	Alice registered	1. POST /auth/login 2. Body: {"email":"alice@example.com","password":"wrongpassword"}	401 Unauthorized, error: "Invalid credentials"	As expected	Wrong password rejected	TC-9.png	PASS
TC-010	Register Bob (Setup)	Server running	1. POST /auth/register 2. Body: {"username":"bob","email":"bob@example.com","password":"pass123"}	201 Created, Bob registered	As expected	Setup successful	TC-10.png	PASS
TC-011	Register Charlie (Setup)	Server running	1. POST /auth/register 2. Body: {"username":"charlie","email":"charlie@example.com","password":"pass123"}	201 Created, Charlie registered	As expected	Setup successful	TC-11.png	PASS
TC-012	Register David (Setup)	Server running	1. POST /auth/register 2. Body: {"username":"david","email":"david@example.com","password":"pass123"}	201 Created, David registered	As expected	Setup successful	TC-12.png	PASS
TC-013	Login Bob (Setup)	Bob registered	1. POST /auth/login 2. Body: {"email":"bob@example.com","password":"pass123"}	200 OK, token saved	As expected	Setup successful	TC-13.png	PASS
TC-014	Login Charlie (Setup)	Charlie registered	1. POST /auth/login 2. Body: {"email":"charlie@example.com","password":"pass123"}	200 OK, token saved	As expected	Setup successful	TC-14.png	PASS

TC-015	Login David (Setup)	David registered	1. POST /auth/login 2. Body: {"email":"david@example.com","password":"pass123"}	200 OK, token saved	As expected	Setup successful	TC-15.png	PASS
TC-016	Get User - Success	Alice logged in	1. GET /users/{aliceId} 2. Header: Authorization: Bearer {aliceToken}	200 OK, user object returned without password fields	As expected	User data retrieved	TC-16.png	PASS
TC-017	Get User - No Token	Alice registered	1. GET /users/{aliceId} 2. No Authorization header	401 Unauthorized, error: "Missing token"	As expected	Auth required	TC-17.png	PASS
TC-018	Get User - Not Found	Alice logged in	1. GET /users/00000000-0000-0000-0000-000000000000 2. With valid token	404 Not Found, error: "User not found"	As expected	Invalid ID handled	TC-18.png	PASS
TC-019	Update Profile - Success	Alice logged in	1. PATCH /users/profile 2. Body: {"username":"alice_updated"} 3. With Alice's token	200 OK, username updated	As expected	Profile updated	TC-19.png	PASS
TC-020	Update Profile - No Token	Server running	1. PATCH /users/profile 2. Body: {"username":"test"} 3. No Authorization header	401 Unauthorized, error: "Missing token"	As expected	Auth required	TC-20.png	PASS
TC-021	Update Profile - Username Taken	Alice, Bob logged in	1. PATCH /users/profile 2. Body: {"username":"bob"} 3. With Alice's token	409 Conflict, error: "Username already taken"	200 OK	update username	TC-21.png	FAIL
TC-022	Update Profile - Email Already Used	Alice, Bob logged in	1. PATCH /users/profile 2. Body: {"email":"alice@example.com"} 3. With Bob's token	409 Conflict, error: "Email already in use"	200 OK	update email	TC-22.png	FAIL

TC-023	Create Team - Success (Team Alpha)	Alice logged in	1. POST /teams 2. Body: {"name":"Team Alpha"} 3. With Alice's token	201 Created, team object with Alice as leader and member	As expected	Team created, teamAlphaId saved	TC-23.png	PASS
TC-024	Create Team - No Token	Server running	1. POST /teams 2. Body: {"name":"Test Team"} 3. No Authorization header	401 Unauthorized, error: "Missing token"	As expected	Auth required	TC-24.png	PASS
TC-025	Create Team - Missing Name	Alice logged in	1. POST /teams 2. Body: {} 3. With Alice's token	400 Bad Request, error: "name is required"	As expected	Validation working	TC-25.png	PASS
TC-026	Create Team Beta (Setup)	Charlie logged in	1. POST /teams 2. Body: {"name":"Team Beta"} 3. With Charlie's token	201 Created, teamBetaId saved	As expected	Setup successful	TC-26.png	PASS
TC-027	Get Team - Success	Team Alpha created	1. GET /teams/{teamAlphaId} 2. With Alice's token	200 OK, complete team object returned	As expected	Team data retrieved	TC-27.png	PASS
TC-028	Get Team - No Token	Team Alpha created	1. GET /teams/{teamAlphaId} 2. No Authorization header	401 Unauthorized, error: "Missing token"	200 OK, team data returned	Auth not enforced	TC-28.png	FAIL
TC-029	Get Team - Not Found	Alice logged in	1. GET /teams/00000000-0000-0000-0000-000000000000 2. With valid token	404 Not Found, error: "Team not found"	As expected	Invalid ID handled	TC-29.png	PASS
TC-030	Add Member - Success (Bob	Alice is Team Alpha leader, Bob registered	1. POST /teams/{teamAlphaId}/members 2. Body: {"userId":"{bobId}"} 3. With Alice's token	200 OK, team with Bob in memberIds	As expected	Member added successfully	TC-30.png	PASS

	to Team Alpha)							
TC-031	Add Member - No Token	Team Alpha created	1. POST /teams/{teamAlphaId}/members 2. Body: {"userId":"{davidId}"} 3. No Authorization header	401 Unauthorized, error: "Missing token"	As expected	Auth required	TC-31.png	PASS
TC-032	Add Member - Team Not Found	Alice logged in	1. POST /teams/00000000-0000-0000-0000-000000000000/members 2. Body: {"userId":"{davidId}"} 3. With valid token	404 Not Found, error: "Team not found"	As expected	Invalid team ID handled	TC-32.png	PASS
TC-033	Add Member - Not Team Leader	Bob is Team Alpha member (not leader)	1. POST /teams/{teamAlphaId}/members 2. Body: {"userId":"{davidId}"} 3. With Bob's token	403 Forbidden, error: "Only the team leader can add members"	As expected	Permission enforced	TC-33.png	PASS
TC-034	Add Member - Missing userId	Alice is Team Alpha leader	1. POST /teams/{teamAlphaId}/members 2. Body: {} 3. With Alice's token	400 Bad Request, error: "userId is required"	As expected	Validation working	TC-34.png	PASS
TC-035	Add Member - User Not Found	Alice is Team Alpha leader	1. POST /teams/{teamAlphaId}/members 2. Body: {"userId":"00000000-0000-0000-0000-000000000000"} 3. With Alice's token	404 Not Found, error: "User not found"	200 OK	shows all IDs	TC-35.png	FAIL
TC-036	Add Member - Already a Member	Bob already in Team Alpha	1. POST /teams/{teamAlphaId}/members 2. Body: {"userId":"{bobId}"} 3. With Alice's token	409 Conflict, error: "User is already a member"	As expected	Duplicate member prevented	TC-36.png	PASS
TC-037	Remove Member - Success	Bob in Team Alpha, Alice is leader	1. DELETE /teams/{teamAlphaId}/members/{bobId} 2. With Alice's token	200 OK, Bob removed from memberIds	As expected	Member removed	TC-37.png	PASS

TC-038	Remove Member - No Token	Team Alpha exists	1. DELETE /teams/{teamAlphaId}/members/{bobId} 2. No Authorization header	401 Unauthorized, error: "Missing token"	As expected	Auth required	TC-38.png	PASS
TC-039	Remove Member - Team Not Found	Alice logged in	1. DELETE /teams/00000000-0000-0000-0000-000000000000/members/{bobId} 2. With valid token	404 Not Found, error: "Team not found"	As expected	Invalid team ID handled	TC-39.png	PASS
TC-040	Remove Member - Not Team Leader	Bob is member (not leader)	1. DELETE /teams/{teamAlphaId}/members/{davidId} 2. With Bob's token (David not member, hypothetical)	403 Forbidden, error: "Only the team leader can remove members"	As expected	Permission enforced	TC-40.png	PASS
TC-041	Remove Member - Cannot Remove Leader	Alice is Team Alpha leader	1. DELETE /teams/{teamAlphaId}/members/{aliceId} 2. With Alice's token	400 Bad Request, error: "Cannot remove the team leader"	As expected	Leader protection working	TC-41.png	PASS
TC-042	Remove Member - Not a Member	David not in Team Alpha	1. DELETE /teams/{teamAlphaId}/members/{davidId} 2. With Alice's token	404 Not Found, error: "User is not a member of this team"	200 OK	shows all IDs	TC-42.png	FAIL
TC-043	Re-add Bob to Team Alpha (Setup)	Bob previously removed	1. POST /teams/{teamAlphaId}/members 2. Body: {"userId":"{bobId}"} 3. With Alice's token	200 OK, Bob re-added	As expected	Setup successful	TC-43.png	PASS
TC-0	Create Project -	Alice is Team Alpha member	1. POST /projects 2. Body: {"name":"Project Alpha","teamId":"{teamAlphaId}"} 3. With Alice's token	201 Created, project with teamId set	As expected	Team project created,	TC-44.png	PASS

44	Success (Team Project)					projectAlphaId saved		
TC-045	Create Project - Success (Open Project)	David logged in	1. POST /projects 2. Body: {"name":"Open Project"} 3. With David's token	201 Created, project with teamId = null	As expected	Open project created, openProjectId saved	TC-45.png	PASS
TC-046	Create Project - No Token	Server running	1. POST /projects 2. Body: {"name":"Test Project"} 3. No Authorization header	401 Unauthorized, error: "Missing token"	As expected	Auth required	TC-46.png	PASS
TC-047	Create Project - Missing Name	Alice logged in	1. POST /projects 2. Body: {} 3. With Alice's token	400 Bad Request, error: "name is required"	As expected	Validation working	TC-47.png	PASS
TC-048	Create Project - Team Not Found	Alice logged in	1. POST /projects 2. Body: {"name":"Test","teamId":"00000000-0000-0000-0000-000000000000"} 3. With valid token	404 Not Found, error: "Team not found"	As expected	Invalid team ID handled	TC-48.png	PASS
TC-049	Create Project - Not a Team Member	David not in Team Alpha	1. POST /projects 2. Body: {"name":"Unauthorized","teamId":"{teamAlphaId}"} 3. With David's token	403 Forbidden, error: "You must be a member of the team to create a project for it"	As expected	Permission enforced	TC-49.png	PASS
TC-050	Get Projects - All	Alice logged in, projects exist	1. GET /projects 2. With Alice's token	200 OK, array with at least 2 projects	As expected	All projects returned	TC-50.png	PASS

TC-051	Get Projects - Filter by Team	Projects exist	1. GET /projects?teamId={teamAlphaId} 2. With Alice's token	200 OK, only Team Alpha projects	As expected	Filtering working	TC-51.png	PASS
TC-052	Get Project - Success	Project Alpha exists	1. GET /projects/{projectAlphaId} 2. With Alice's token	200 OK, complete project object	As expected	Project data retrieved	TC-52.png	PASS
TC-053	Get Project - No Token	Project exists	1. GET /projects/{projectAlphaId} 2. No Authorization header	401 Unauthorized, error: "Missing token"	As expected	Auth required	TC-53.png	PASS
TC-054	Get Project - Not Found	Alice logged in	1. GET /projects/00000000-0000-0000-0000-000000000000 2. With valid token	404 Not Found, error: "Project not found"	As expected	Invalid ID handled	TC-54.png	PASS
TC-055	Create Issue - Success (Team Project)	David logged in, Project Alpha exists	1. POST /projects/{projectAlphaId}/issues 2. Body: {"title":"Login broken","description":"Firefox issue"} 3. With David's token	201 Created, issue with status "open", activityLog with "opened" entry	As expected	Issue created, issueAlpha1Id saved	TC-55.png	PASS
TC-056	Create Issue - Success (Open Project)	Charlie logged in, Open Project exists	1. POST /projects/{openProjectId}/issues 2. Body: {"title":"UI misalignment","description":"Buttons issue"} 3. With Charlie's token	201 Created, issue on open project	As expected	Issue created, issueOpen1Id saved	TC-56.png	PASS
TC-057	Create Issue - No Token	Project exists	1. POST /projects/{projectAlphaId}/issues 2. Body: {"title":"Test","description":"Test"} 3. No Authorization header	401 Unauthorized, error: "Missing token"	As expected	Auth required	TC-57.png	PASS
TC-0	Create Issue - Proje	Alice logged in	1. POST /projects/00000000-0000-0000-0000-000000000000/issues 2. Body: {"title":"Test","description":"Test"} 3. With valid token	404 Not Found, error: "Project not found"	As expected	Invalid project ID handled	TC-58.png	PASS

58	ct Not Found							
TC-059	Create Issue - Missing Fields	Alice logged in	1. POST /projects/{projectAlphaId}/issues 2. Body: {"title":"Only title"} 3. With Alice's token	400 Bad Request, error: "title and description are required"	As expected	Validation working	TC-59.png	PASS
TC-060	Get Project Issues - Success	Issues exist on Project Alpha	1. GET /projects/{projectAlphaId}/issues 2. With Alice's token	200 OK, array of issues for project	As expected	Issues retrieved	TC-60.png	PASS
TC-061	Get Project Issues - No Token	Project exists	1. GET /projects/{projectAlphaId}/issues 2. No Authorization header	401 Unauthorized, error: "Missing token"	As expected	Auth required	TC-61.png	PASS
TC-062	Get Project Issues - Project Not Found	Alice logged in	1. GET /projects/00000000-0000-0000-0000-000000000000/issues 2. With valid token	404 Not Found, error: "Project not found"	As expected	Invalid ID handled	TC-62.png	PASS
TC-063	Get Issues - All	Alice logged in, issues exist	1. GET /issues 2. With Alice's token	200 OK, array of all issues	As expected	All issues returned	TC-63.png	PASS
TC-064	Get Issues - Filter by Status	Issues with various statuses exist	1. GET /issues?status=open 2. With Alice's token	200 OK, only open issues	As expected	Status filtering working	TC-64.png	PASS
TC-065	Get Issues - No Token	Issues exist	1. GET /issues 2. No Authorization header	401 Unauthorized, error: "Missing token"	As expected	Auth required	TC-65.png	PASS

TC-066	Get Issue - Success	Issue exists	1. GET /issues/{issueAlpha1Id} 2. With Alice's token	200 OK, complete issue with resolveHistory and activityLog	As expected	Issue data retrieved	TC-66.png	PASS
TC-067	Get Issue - No Token	Issue exists	1. GET /issues/{issueAlpha1Id} 2. No Authorization header	401 Unauthorized, error: "Missing token"	As expected	Auth required	TC-67.png	PASS
TC-068	Get Issue - Not Found	Alice logged in	1. GET /issues/00000000-0000-0000-0000-000000000000 2. With valid token	404 Not Found, error: "Issue not found"	As expected	Invalid ID handled	TC-68.png	PASS
TC-069	Start Issue - Success (Team Member)	Bob in Team Alpha, issue open on Project Alpha	1. PATCH /issues/{issueAlpha1Id}/start 2. With Bob's token	200 OK, status="in_progress", resolverId=Bob, resolveHistory entry	As expected	Issue started by team member	TC-69.png	PASS
TC-070	Start Issue - Success (Open Project)	Charlie logged in, issue open on Open Project	1. PATCH /issues/{issueOpen1Id}/start 2. With Charlie's token	200 OK, status="in_progress", resolverId=Charlie	As expected	Issue started on open project	TC-70.png	PASS
TC-071	Start Issue - No Token	Issue open	1. PATCH /issues/{issueAlpha1Id}/start 2. No Authorization header	401 Unauthorized, error: "Missing token"	As expected	Auth required	TC-71.png	PASS
TC-072	Start Issue - Issue Not Found	Alice logged in	1. PATCH /issues/00000000-0000-0000-0000-000000000000/start 2. With valid token	404 Not Found, error: "Issue not found"	As expected	Invalid ID handled	TC-72.png	PASS
TC-073	Start Issue - Not Open	Issue already in_progress	1. PATCH /issues/{issueAlpha1Id}/start 2. With Alice's token	400 Bad Request, error: "Only open issues can be started"	200 OK	not an open issue but still started	TC-73.png	FAIL

TC-074	Create Issue for Permission Test (Setup)	Alice logged in	1. POST /projects/{projectAlphaId}/issues 2. Body: {"title":"Permission test","description":"Test"} 3. With Alice's token	201 Created, issuePermTestId saved	As expected	Setup successful	TC-74.png	PASS
TC-075	Start Issue - Not Team Member	Charlie not in Team Alpha, issue on Project Alpha	1. PATCH /issues/{issuePermTestId}/start 2. With Charlie's token	403 Forbidden, error: "You do not have permission to resolve this issue"	As expected	Permission enforced	TC-75.png	PASS
TC-076	Abandon Issue - Success	Bob is current resolver of issueAlpha1Id (in_progress)	1. PATCH /issues/{issueAlpha1Id}/abandon 2. With Bob's token	200 OK, status="open", resolverId=null, resolveHistory outcome="abandoned"	403 forbidden	(cascading effect of DEF-006)	TC-76.png	FAIL
TC-077	Abandon Issue - No Token	Issue in_progress	1. PATCH /issues/{issueAlpha1Id}/abandon 2. No Authorization header	401 Unauthorized, error: "Missing token"	As expected	Auth required	TC-77.png	PASS
TC-078	Abandon Issue - Issue Not Found	Bob logged in	1. PATCH /issues/00000000-0000-0000-0000-000000000000/abandon 2. With valid token	404 Not Found, error: "Issue not found"	As expected	Invalid ID handled	TC-78.png	PASS
TC-079	Abandon Issue - Not In Progress	Issue in "open" status	1. PATCH /issues/{issueAlpha1Id}/abandon 2. With Bob's token	400 Bad Request, error: "Only in-progress issues can be abandoned"	403 forbidden	(cascading effect of DEF-006)	TC-79.png	FAIL
TC-080	Bob Starts Issue Again (Setup)	Issue in open status	1. PATCH /issues/{issueAlpha1Id}/start 2. With Bob's token	200 OK, issue started	As expected	Setup successful	TC-80.png	PASS
TC-0	Abandon Issue - Not	Alice is not resolver of issueAlpha1Id (Bob is)	1. PATCH /issues/{issueAlpha1Id}/abandon 2. With Alice's token	403 Forbidden, error: "Only the current resolver	As expected	Permission enforced	TC-81.png	PASS

81	Current Resolver			can abandon this issue"				
TC-082	Submit Issue - Success	Bob is current resolver (in_progress)	1. PATCH /issues/{issueAlpha1Id}/submit 2. With Bob's token	200 OK, status="resolved", resolveHistory outcome="submitted", resolvedBy=Bob	As expected	Solution submitted	TC-82.png	PASS
TC-083	Submit Issue - No Token	Issue in_progress	1. PATCH /issues/{issueAlpha1Id}/submit 2. No Authorization header	401 Unauthorized, error: "Missing token"	As expected	Auth required	TC-83.png	PASS
TC-084	Submit Issue - Issue Not Found	Bob logged in	1. PATCH /issues/00000000-0000-0000-0000-000000000000/submit 2. With valid token	404 Not Found, error: "Issue not found"	As expected	Invalid ID handled	TC-84.png	PASS
TC-085	Submit Issue - Not In Progress	Issue in "resolved" status (already submitted)	1. PATCH /issues/{issueAlpha1Id}/submit 2. With Bob's token	400 Bad Request, error: "Only in-progress issues can be submitted"	200 OK	not a progress issue but still submitted	TC-85.png	FAIL
TC-086	Create Issue for Submit Test (Setup)	David logged in	1. POST /projects/{projectAlphaId}/issues 2. Body: {"title":"Submit test","description":"Test"} 3. With David's token	201 Created, issueSubmitTestId saved	As expected	Setup successful	TC-86.png	PASS
TC-087	Alice Starts Submit Test Issue (Setup)	Alice logged in, issue open	1. PATCH /issues/{issueSubmitTestId}/start 2. With Alice's token	200 OK, issue started	As expected	Setup successful	TC-87.png	PASS
TC-0	Submit Issue - Not Curr	Bob is not resolver of issueSubmitTestId (Alice is)	1. PATCH /issues/{issueSubmitTestId}/submit 2. With Bob's token	403 Forbidden, error: "Only the current resolver can submit this issue"	200 OK	not a current resolver but still	TC-88.png	FAIL

88	ent Resolver					issue submitted		
TC-089	Alice Submits Issue (Setup)	Alice is resolver	1. PATCH /issues/{issueSubmitTestId}/submit 2. With Alice's token	200 OK, issue submitted	As expected	Setup successful	TC-89.png	PASS
TC-090	Accept Issue - Success	David is creator of issueAlpha1Id (resolved)	1. PATCH /issues/{issueAlpha1Id}/accept 2. With David's token	200 OK, status="closed", resolveHistory outcome="accepted"	As expected	Solution accepted	TC-90.png	PASS
TC-091	Accept Issue - No Token	Issue resolved	1. PATCH /issues/{issueSubmitTestId}/accept 2. No Authorization header	401 Unauthorized, error: "Missing token"	As expected	Auth required	TC-91.png	PASS
TC-092	Accept Issue - Issue Not Found	David logged in	1. PATCH /issues/00000000-0000-0000-0000-000000000000/accept 2. With valid token	404 Not Found, error: "Issue not found"	As expected	Invalid ID handled	TC-92.png	PASS
TC-093	Accept Issue - Not Resolved	Issue in "open" status (issuePermTestId)	1. PATCH /issues/{issuePermTestId}/accept 2. With Alice's token	400 Bad Request, error: "Only resolved issues can be accepted"	500 internal server error	Showing code snippets	TC-93.png	FAIL
TC-094	Accept Issue - Not Issue Creator	Bob is not creator of issueSubmitTestId (David is)	1. PATCH /issues/{issueSubmitTestId}/accept 2. With Bob's token	403 Forbidden, error: "Only the issue creator can accept a solution"	As expected	Permission enforced	TC-94.png	PASS
TC-095	Create Issue for Reopen Test (Setup)	Alice logged in	1. POST /projects/{projectAlphaId}/issues 2. Body: {"title":"Reopen test","description":"Test"} 3. With Alice's token	201 Created, issueReopenTestId saved	As expected	Setup successful	TC-95.png	PASS
TC-0	Bob Starts Reop	Bob logged in, issue open	1. PATCH /issues/{issueReopenTestId}/start 2. With Bob's token	200 OK, issue started	As expected	Setup successful	TC-96.png	PASS

96	en Test Issue (Setup)							
TC-097	Bob Submits Reopen Test Issue (Setup)	Bob is resolver	1. PATCH /issues/{issueReopenTestId}/submit 2. With Bob's token	200 OK, issue submitted	As expected	Setup successful	TC-97.png	PASS
TC-098	Reopen Issue - Success (From Resolved)	Alice is creator, issue resolved	1. PATCH /issues/{issueReopenTestId}/reopen 2. With Alice's token	200 OK, status="open", resolverId=null, reopenCount=1, outcome="reopened"	As expected	Issue reopened from resolved	TC-98.png	PASS
TC-099	Reopen Issue - Success (From Closed)	David is creator of issueAlpha1Id (closed)	1. PATCH /issues/{issueAlpha1Id}/reopen 2. With David's token	200 OK, status="open", resolverId=null, reopenCount=1	As expected	Issue reopened from closed	TC-99.png	PASS
TC-100	Reopen Issue - No Token	Issue closed/resolved	1. PATCH /issues/{issueAlpha1Id}/reopen 2. No Authorization header	401 Unauthorized, error: "Missing token"	As expected	Auth required	TC-100.png	PASS
TC-101	Reopen Issue - Issue Not Found	Alice logged in	1. PATCH /issues/00000000-0000-0000-0000-000000000000/reopen 2. With valid token	404 Not Found, error: "Issue not found"	As expected	Invalid ID handled	TC-101.png	PASS
TC-102	Reopen Issue - Not Resolved or	Issue in "open" status (already reopened)	1. PATCH /issues/{issueAlpha1Id}/reopen 2. With David's token	400 Bad Request, error: "Only resolved or closed issues can be reopened"	As expected	State validation working	TC-102.png	PASS

	Closed							
TC-103	Bob Starts Issue for Not Creator Test (Setup)	Bob logged in, issue open	1. PATCH /issues/{issueAlpha1Id}/start 2. With Bob's token	200 OK, issue started	As expected	Setup successful	TC-103.png	PASS
TC-104	Bob Submits Issue (Setup)	Bob is resolver	1. PATCH /issues/{issueAlpha1Id}/submit 2. With Bob's token	200 OK, issue submitted	As expected	Setup successful	TC-104.png	PASS
TC-105	Reopen Issue - Not Issue Creator	Bob is not creator of issueAlpha1Id (David is)	1. PATCH /issues/{issueAlpha1Id}/reopen 2. With Bob's token	403 Forbidden, error: "Only the issue creator can reopen an issue"	As expected	Permission enforced	TC-105.png	PASS
TC-106	Reopen Issue for Comments (Setup)	David is creator	1. PATCH /issues/{issueAlpha1Id}/reopen 2. With David's token	200 OK, issue reopened	As expected	Setup successful	TC-106.png	PASS
TC-107	Post Comment - Success (Top Level)	Alice logged in, issue open/in_progress/resolved	1. POST /issues/{issueAlpha1Id}/comments 2. Body: {"body":"Reproduce on Chrome"} 3. With Alice's token	201 Created, comment with parentCommentId=null, comment1Id saved	As expected	Top-level comment posted	TC-107.png	PASS
TC-108	Post Comment - Success (Nested Reply)	Bob logged in, comment1 exists	1. POST /issues/{issueAlpha1Id}/comments 2. Body: {"body":"Reply","parentCommentId":"{comment1Id}"} 3. With Bob's token	201 Created, comment with parentCommentId=comment1Id, comment2Id saved	As expected	Nested reply posted	TC-108.png	PASS

TC-109	Post Comment - Success (Deep Nesting)	Charlie logged in, comment2 exists	1. POST /issues/{issueAlpha1Id}/comments 2. Body: {"body":"Deep reply","parentCommentId":"{comment2Id}"} 3. With Charlie's token	201 Created, deeply nested comment	As expected	Deep nesting allowed	TC-109.png	PASS
TC-110	Post Comment - No Token	Issue exists	1. POST /issues/{issueAlpha1Id}/comments 2. Body: {"body":"Test"} 3. No Authorization header	401 Unauthorized, error: "Missing token"	As expected	Auth required	TC-110.png	PASS
TC-111	Post Comment - Issue Not Found	Alice logged in	1. POST /issues/00000000-0000-0000-0000-000000000000/comments 2. Body: {"body":"Test"} 3. With valid token	404 Not Found, error: "Issue not found"	As expected	Invalid issue ID handled	TC-111.png	PASS
TC-112	Start Issue for Closed Test (Setup)	Bob logged in, issue open	1. PATCH /issues/{issueAlpha1Id}/start 2. With Bob's token	200 OK, issue started	As expected	Setup successful	TC-112.png	PASS
TC-113	Bob Submits for Closed Test (Setup)	Bob is resolver	1. PATCH /issues/{issueAlpha1Id}/submit 2. With Bob's token	200 OK, issue submitted	As expected	Setup successful	TC-113.png	PASS
TC-114	David Accepts for Closed Test (Setup)	David is creator	1. PATCH /issues/{issueAlpha1Id}/accept 2. With David's token	200 OK, issue closed	As expected	Setup successful	TC-114.png	PASS
TC-1	Post Comment - Issue	Issue in "closed" status	1.POST /issues/{issueAlpha1Id}/comments 2. Body: {"body":"Comment on closed"} 3. With Alice's token	400 Bad Request, error: "Cannot comment on closed issues"	201 Created, comm men	Closed issue restriction not	TC-115.png	FAIL

15	Closed				t posted	enforced		
TC-116	Post Comment - Missing Body	Alice logged in, issue open	1. POST /issues/{issueOpen1Id}/comments 2. Body: {} 3. With Alice's token	400 Bad Request, error: "body is required"	As expected	Validation working	TC-116.png	PASS
TC-117	Post Comment - Parent Not Found	Alice logged in, issue exists	1. POST /issues/{issueOpen1Id}/comments 2. Body: {"body":"Reply","parentCommentId":"00000000-0000-0000-0000-000000000000"} 3. With Alice's token	404 Not Found, error: "Parent comment not found"	201 created	Comment posted	TC-117.png	FAIL
TC-118	Create Comment on Open Project (Setup)	David logged in, issueOpen1Id exists	1. POST /issues/{issueOpen1Id}/comments 2. Body: {"body":"Comment on open"} 3. With David's token	201 Created, commentOpenProjectId saved	As expected	Setup successful	TC-118.png	PASS
TC-119	Post Comment - Parent from Different Issue	Alice logged in, commentOpenProjectId from issueOpen1Id	1. POST /issues/{issueReopenTestId}/comments 2. Body: {"body":"Cross-issue reply","parentCommentId":"{commentOpenProjectId}"} 3. With Alice's token	400 Bad Request, error: "Parent comment does not belong to this issue"	201 Created, cross-issue reply allowed	Parent validation missing	TC-119.png	FAIL
TC-120	Get Comments - Success (Nested Tree)	Comments exist on issueOpen1Id	1.GET /issues/{issueOpen1Id}/comments 2. With Alice's token	200 OK, nested tree with replies arrays	As expected	Nested structure returned	TC-120.png	PASS
TC-121	Get Comments - No	Comments exist	1. GET /issues/{issueOpen1Id}/comments 2. No Authorization header	401 Unauthorized, error: "Missing token"	As expected	Auth required	TC-121.png	PASS

	Token							
TC-122	Get Comments - Issue Not Found	Alice logged in	1. GET /issues/00000000-0000-0000-0000-000000000000/comments 2. With valid token	404 Not Found, error: "Issue not found"	As expected	Invalid ID handled	TC-122.png	PASS

5. Defect Reports

Summary of Defects

Severity	Count	Defect IDs
Critical	5	DEF-001, DEF-002, DEF-007, DEF-008, DEF-011
High	6	DEF-003, DEF-005, DEF-006, DEF-009, DEF-012, DEF-014
Medium	3	DEF-004, DEF-010, DEF-013
TOTAL	**14**	

5.1 Critical Defects

Defect ID	Severity	Test Case	Title	Description / Steps to Reproduce	Expected Result	Actual Result	Impact
DEF-001	Critical	TC-021	Duplicate Username Update	Alice tries to change her username to "bob" (already taken) via PATCH /users/profile	409 Conflict + proper error message	200 OK - update username	Misleading API response
DEF-002	Critical	TC-022	Duplicate Email Update	Bob tries to change his email to alice@example.com via PATCH /users/profile	409 Conflict + proper error message	200 OK - update email	Misleading API response
DEF-003	High	TC-028	Authentication Bypass on GET /teams/:id	Request team details without Authorization header	401 Unauthorized	200 OK - full team data returned	Sensitive team structure leaked without login
DEF-004	Medium	TC-042	Silent Success on Removing Non-Member	Alice tries to remove David (never added) from Team Alpha	404 Not Found	200 OK shows all IDs	Misleading API response, client confusion
DEF-005	High	TC-035	Server Crash on Invalid userId in Add Member	Alice tries to add userId = 00000000-… (non-existent) to team	404 Not Found	200 OK shows all IDs	Service disruption, DoS risk
DEF-006	High	TC-073	Duplicate Start Allowed – Resolver Hijacking	Alice starts an issue that Bob already started (in_progress)	400 Bad Request	200 OK - not an open issue but still started	Work ownership stolen, audit trail corrupted
DEF-007	Critical	TC-088	Unauthorized Issue Submission Allowed	Bob submits an issue that Alice is currently resolving	403 Forbidden	200 OK – not a current resolver but	Complete breakdown of resolver ownership

						still issue submitted	
DEF-008	Critical	TC-093	Accept Non-Resolved Issue Allowed	Alice accepts an issue that is still “open”	400 Bad Request	500 Internal Server Error	Complete service disruption, production blocker
DEF-009	High	TC-085	Duplicate Submission Allowed	Bob submits an issue that is already “resolved”	400 Bad Request	200 OK - not a progress issue but still submitted	Resolve history polluted, metrics wrong
DEF-010	Medium	TC-076, TC-079	Abandon Fails Due to State Corruption	Bob tries to abandon after DEF-006 corrupted state	Should work (if in_progress)	403 forbidden (cascading effect of DEF-006)	Users cannot abandon corrupted issues
DEF-011	Critical	TC-115	Comments Allowed on Closed Issues	Alice posts comment on a closed issue	400 Bad Request - “Cannot comment on closed issues”	201 Created - comment posted	Closed issues never truly closed, spam risk
DEF-012	High	TC-117	Server Crash on Invalid Parent Comment ID	Reply with parentCommentId = 00000000-… (does not exist)	404 Not Found	201 created comment	Service disruption
DEF-013	Medium	N/A (setup)	Default JWT_SECRET Used in Production Risk	JWT_SECRET remains “dev_secret” in production	Warning or refusal to start	Silent acceptance	All tokens can be forged – complete auth bypass
DEF-014	High	TC-119	Cross-Issue Parent Comment Allowed	Reply to a comment from a completely different issue	400 Bad Request - parent must belong to same issue	201 Created - cross-issue reply succeeds	Comment tree corrupted, frontend will crash

6. Individual Reflections

Student ID: 011221521

I was responsible for testing the Authentication module (TC-01 through TC-15) and the Projects module (TC-44 through TC-62), Issues module (TC-63 to TC-106).

Student ID: 011221520

I was responsible for testing the My focus was on Teams Module (TC-25 through TC-43), Issues module (TC-63 to TC-106), Comments Module (TC-107 to TC-122).

Student ID: 011221444

I was responsible for testing as well as the Users module (TC-16 through TC-22), Issues module (TC-63 to TC-106), Comments Module (TC-107 to TC-122).

Student ID: 011223697

I was responsible for testing the Teams module (TC-25 through TC-43) together with Projects module (TC-44 through TC-62), Issues module (TC-63 to TC-106).

Appendix A: Collection Variables Reference

The following Postman collection variables were maintained throughout testing:

Base Configuration:

baseUrl: http://localhost:3000

User Identifiers:

aliceId, bobId, charlieId, davidId: User UUIDs

aliceToken, bobToken, charlieToken, davidToken: JWT authentication tokens

Team Identifiers:

teamAlphaId: Alice's team (with Bob as member)

teamBetaId: Charlie's independent team

Project Identifiers:

projectAlphaId: Team-restricted project (Team Alpha)

openProjectId: Open project (no team restriction)

Issue Identifiers:

issueAlpha1Id: Main test issue (David creator, Bob resolver)

issueOpen1Id: Open project issue

issuePermTestId: Permission validation issue

issueSubmitTestId: Submit workflow test issue

issueReopenTestId: Reopen workflow test issue

Comment Identifiers:

comment1Id: Top-level comment

comment2Id: First-level reply

commentOpenProjectId: Cross-issue validation comment